Deep Learning – 1RT720

Report for Hand-in Assignment 2

Hugo Eklöf

November 3, 2026

Exercise 1. *Multi-layer fully-connected neural network.*

In exercise 1, we implemented the exact same fully connected network as in assignment 1 but with Pytorch functions. The learning parameters were also set to the exact same with activation function ReLU, which was:

- Number of hidden layers = 2
- Number of nodes per layer = 120
- Batch size = 30
- Learning rate = 0.01
- Number of epochs = 20

The model training ran on Google Colab's T4-processor (this will be the case for every exercise in this assignment). The results for each epoch can be seen in Figure 1. As we can see, the final test cost is 0.0808, which is very close to 0.0797 from assignment 1. Simultaneously, the test accuracy increased to 98.20% from 97.48%. We can also see in the accuracy graph that the test accuracy started to stabilize above 97% after about 7 epochs already, and could have been interrupted with a tolerance criteria (similar to assignment 1). But for the sake of keeping the number of epochs consistent for both assignments, this was not implemented. Since it had to run all 20 epochs, this also resulted in significantly longer runtime (5 min, 1.4 sec instead of 1 min, 42.9 sec). Other than that, the performances of the two methods were quite similar.

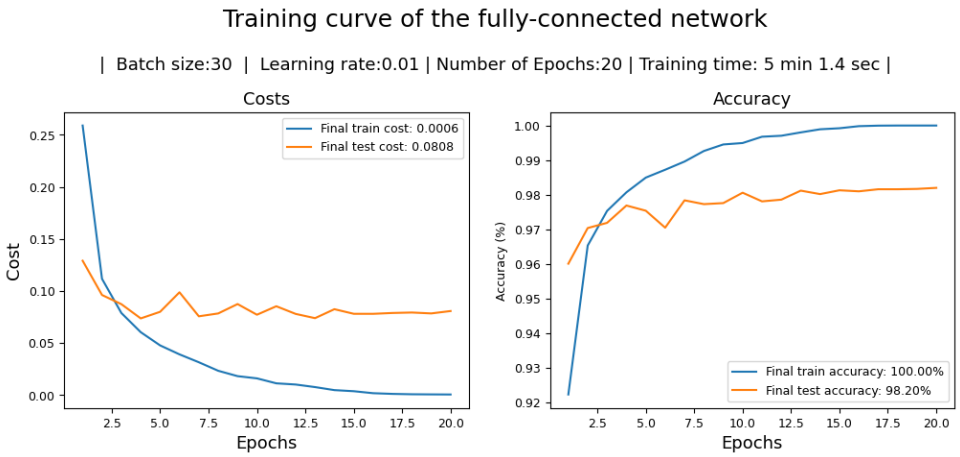


Figure 1: Train/test costs/accuracies per epoch for the fully-connected network.

The number of learnable parameters could be calculated in the code directly (see Appendix for details) and can be seen in Table 1. These are to be compared to the number of parameters in exercise 2.

Layer	**Weights**	**Biases**	**Total**
FC1	94080	120	94200
FC2	14400	120	14520
FC3	1200	10	1210
Total	109680	250	109930

Table 1: Number of learnable parameters per layer of the fully connected network.

Exercise 2 *Multi-layer convolutional neural network.*

Next, we implemented a CNN with 3 convolutional layers and one fully connected layer (the hyper-parameters were given in the instructions). Contrary to the last exercise, we here add a stop criteria similar to assignment 1 (in particular, stop if the average train cost does not decrease with more than 0.005 units for 5 full epochs). It is worth noting again that a separate validation data set would be significantly better for constructing a stop criteria, but for our purposes in this assignment, the training set will suffice. With a batch size of 30 and learning rate of 0.01, the model converged to a test accuracy above 98% after just 10 epochs (see Figure 2).

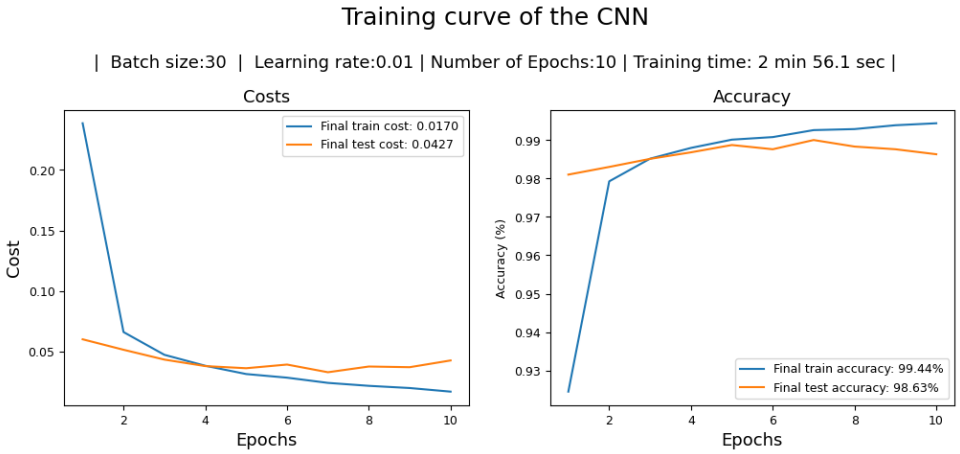


Figure 2: Train/test costs/accuracies per epoch for the CNN.

Layer	**Weights**	**Biases**	**Total**
Conv1	72	8	80
Conv2	1152	16	1168
Conv3	4608	32	4640
FC	15680	10	15690
Total	21512	66	21578

Table 2: Number of learnable parameters per layer of the CNN.

The fully connected network had a total of 109930 learnable parameters (see Table 1), which is more than five times greater than the total for the CNN. This result is expected since the number of weights and biases for the CNN does not depend on the spatial resolution of the feature maps, while the fully connected network does. This is because the same kernel is applied across the entire image, training the same set of parameters.

Exercise 3. *Use the ADAM optimizer.*

Next, we try and change the optimizer from SGD to ADAM with standard parameters (GradientDecayFactor ($\beta 1$) : 0.9000, SquaredGradientDecayFactor ($\beta 2$) : 0.9990, ϵ : 10^{-8}, and a learning rate of 0.001). The final results seen in Figure 3 show that the test accuracy did first reach 99% after about 6 epochs, which is an improvement compared to when SGD was used in exercise 2.

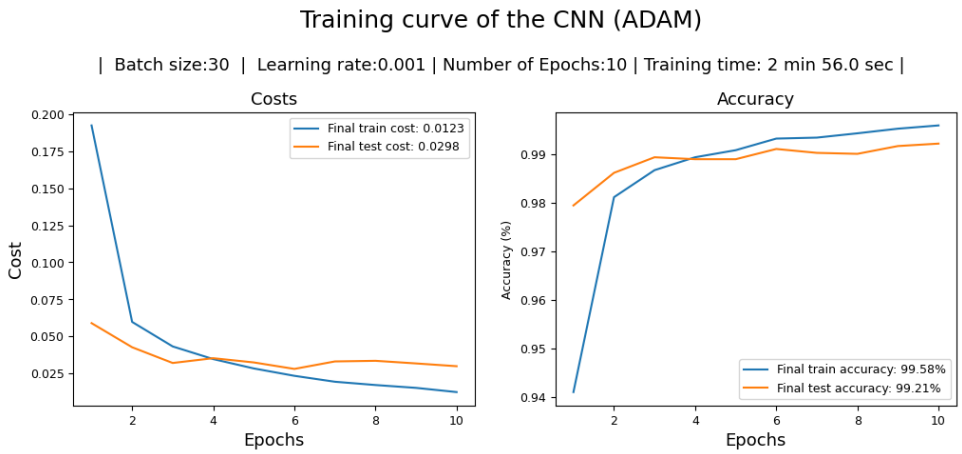


Figure 3: Train/test costs/accuracies per epoch for the CNN with the ADAM optimizer.

Exercise 4. *Residual connections.*

Next, we add a residual connection to the CNN. The max poolings were removed since we'll add the data input after all three convolutions and wish to preserve the data structure to the extent possible (without changing our convolutional layer's hyperparameters). Still, a projection function was added to allow the input to match the dimensions of the output given by the residual block. The fully connected layer was also reshaped accordingly. Figure 4 shows that this model converged after 12 full epochs, which is slower compared to the model without residual connection. Furthermore, the test accuracy does not reach above 99%, and so it's safe to conclude that this residual connection does not improve performance. Had the network been a lot deeper, it might have benefited from residual connections, but since it's quite shallow, the residual connection probably just adds unnecessary noise.

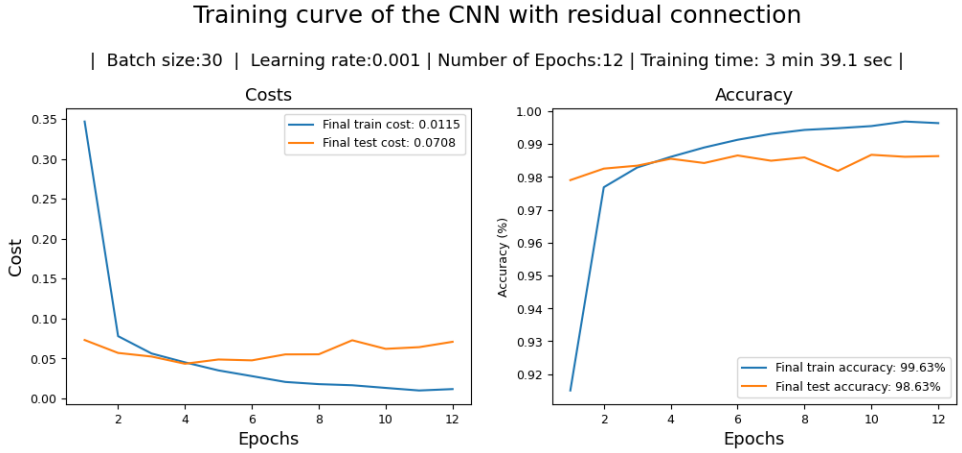


Figure 4: Train/test costs/accuracies per epoch for the CNN with a residual connection and the ADAM optimizer.

In order to see whether this model suffers from overfitting or underfitting, the same code was ran for 30 full epochs without the stop criteria. As we can see in Figure 5, the test cost shows a clear trend toward larger values as the number of epochs increases, which indicates overfitting.

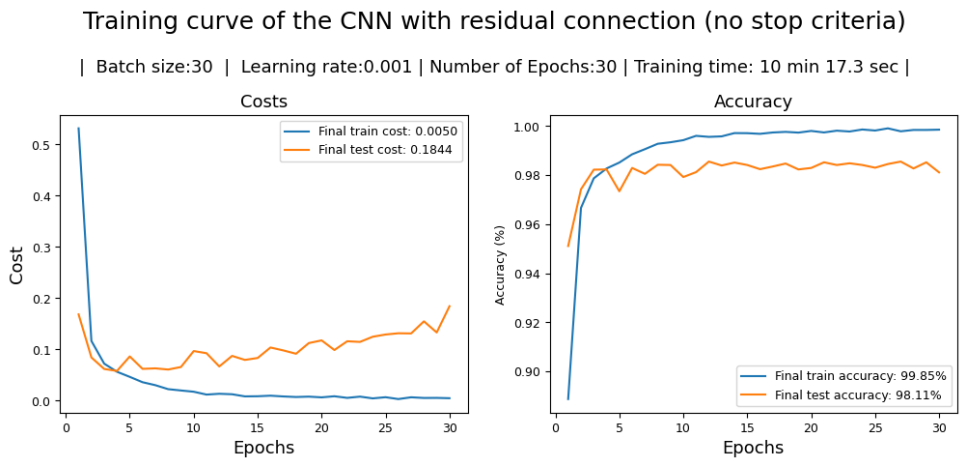


Figure 5: Train/test costs/accuracies per epoch for the CNN with a residual connection and the ADAM optimizer without stop criteria.

Exercise 5. *Improve the CNN with Regularization.*

In the final exercise, we implemented two different regularization techniques to our last CNN independently: Batch normalization and dropout. See Figure 6 for the results with batch normalization and Figure 7 for the results with dropout ($p = 0.5$).

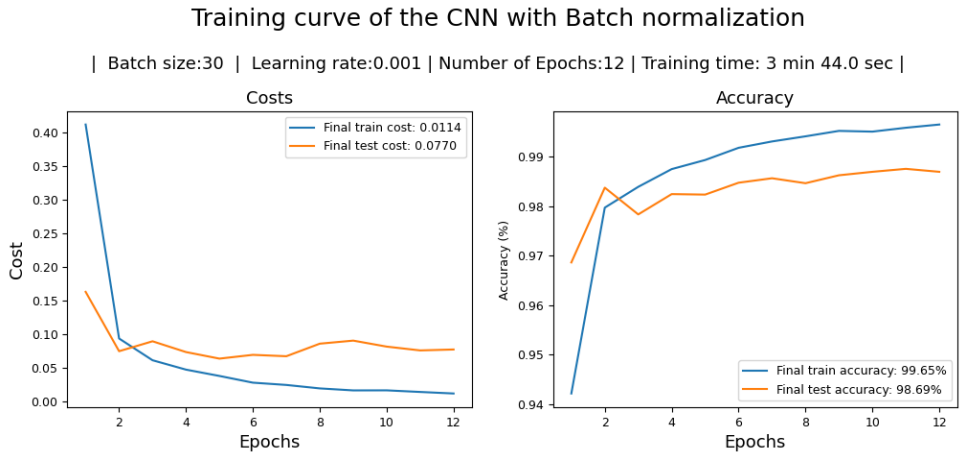


Figure 6: Train/test costs/accuracies per epoch for the CNN with batch normalization.

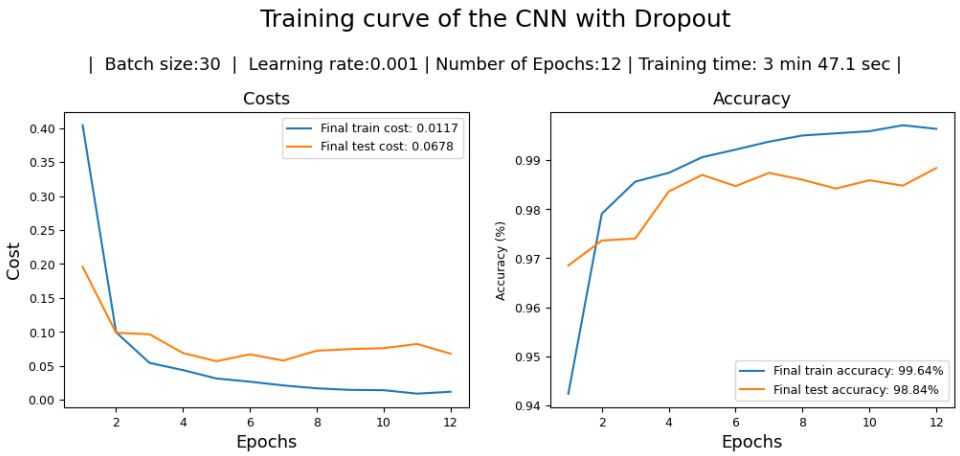


Figure 7: Train/test costs/accuracies per epoch for the CNN with dropout.

While the results ended up being quite similar, the model with dropout implemented was chosen since it got a slightly higher final test accuracy and lower test cost compared to the model with batch normalization. For the dropout model, we generated a confusion matrix (see Figure 8) and 10 images that were misclassified, one for each number 0-9 (see Figure 9). Due to how "sloppily" most of the numbers are written in the misclassified images, it is quite understandable that the model got these wrong.

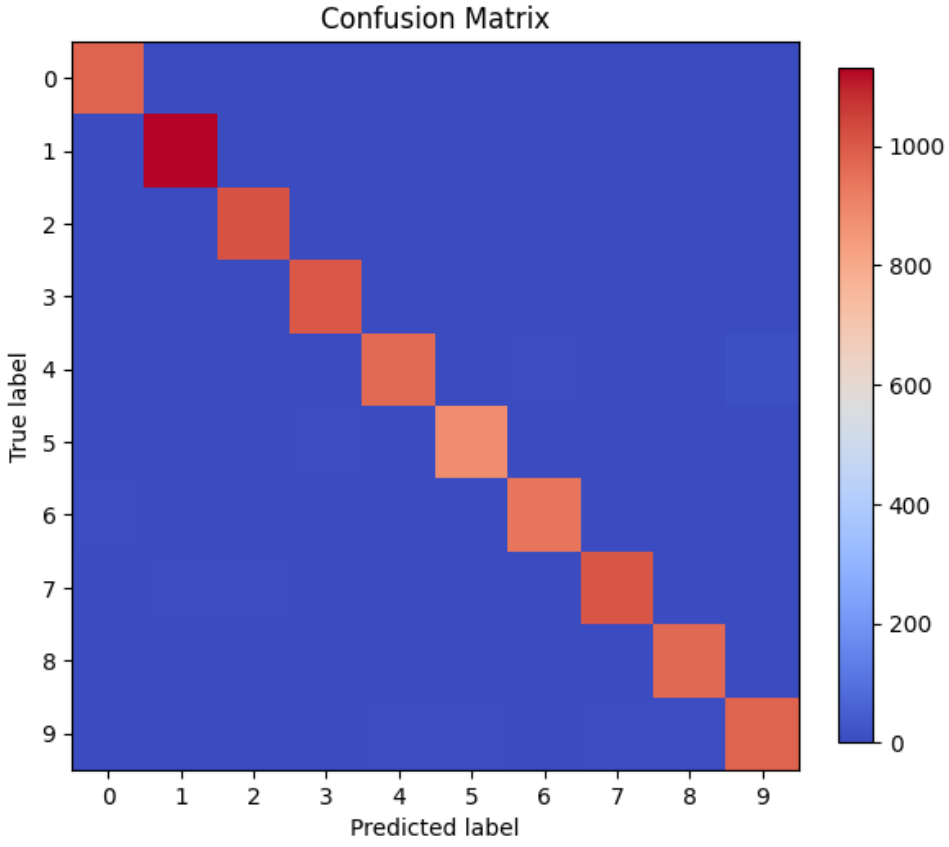


Figure 8: A confusion matrix for the model with dropout.

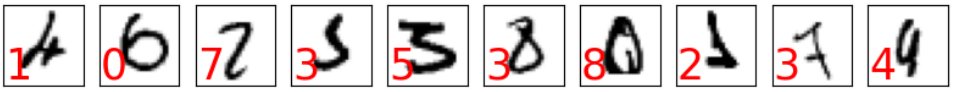


Figure 9: 10 misclassified images. The true values are (from left to right) 4, 6, 2, 5, 3, 8, 0, 1, 7, 9. The predicted values are shown in red.

Use of generative AI

Generative AI was used to...

- debug code that produced error messages that were hard to interpret. The suggested fixes were thoroughly analyzed before they were implemented.
- understand different Pytorch commands, e.g. what inputs "nn.Conv2d" needed.
- gain some intuitional insight into some of the underlying theory, e.g. advantages and disadvantages of using residual connections.
- provide LaTeX code that enabled line breaking in minted.

☒ The report generally and transparently details all uses of AI-tools, e.g., as a minium by explaining how it was used and to what extend (see the acknowledgment for an example).

☒ Text or code proposed or modified by an AI has been checked for correctness and that the meaning is as intended.

☒ I take full responsibility for the whole text.

A Code

```
import math
import numpy as np
import torch
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import time

from torch.utils.data import DataLoader
from torchvision.datasets import MNIST
from torchvision.utils import make_grid
from torchvision.transforms import v2 as T

seed = 123
np.random.seed(seed)
torch.manual_seed(seed)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# T.Compose takes a list of transformations and applies them in sequence
transform = T.Compose([
    T.PILToTensor(),
    T.ToDtype(torch.float32, scale=True), # scale=True scales values to the interval [0,1] if needed
])

trainset = MNIST(
    './data',
    train=True,
    download=True,
    transform=transform)

testset = MNIST(
    './data',
    train=False,
    download=True,
    transform=transform)

# extract a complete PyTorch dataset
def extract(dataset):
    datasize = len(dataset)
    dataloader = torch.utils.data.DataLoader(dataset, batch_size=datasize, shuffle=False)
    return next(iter(dataloader))

# extract all test images and labels into PyTorch tensors
# the training data will be loaded in batches during training
test_X, test_Y = extract(testset)

def weight_init(net_l):
    # network initialization function
    if not isinstance(net_l, list):
        net_l = [net_l]
    for net in net_l:
        for m in net.modules():
            if isinstance(m, nn.Linear):
```

```
                nn.init.kaiming_uniform_(m.weight)
                m.bias.data.fill_(0.0)

class NetFullyConnected(nn.Module):
    def __init__(self):
        super(NetFullyConnected, self).__init__()
        U1 = 120 # number of unit for hidden layer 1
        U2 = 120 # number of unit for hidden layer 2

        # Neural layers
        self.fc1 = nn.Linear(784, U1)
        self.fc2 = nn.Linear(U1, U2)
        self.fc3 = nn.Linear(U2, 10)
        self.activate = nn.ReLU()

        # Network initialization
        self.apply(weight_init)

    def forward(self, input):
        # flatten the data into a matrix with 28 x 28 = 784 columns
        x = input.view(-1, 784)

        # compute hidden layers and predictions
        x = self.activate(self.fc1(x))
        x = self.activate(self.fc2(x))
        x = self.fc3(x)

        return x

class NetCNN(nn.Module):
    def __init__(self):
        super(NetCNN, self).__init__()

        # stride=1 is default
        self.conv1 = nn.Conv2d(1, 8, (3,3), padding=1)
        self.conv2 = nn.Conv2d(8, 16, (3,3), padding=1)
        self.conv3 = nn.Conv2d(16, 32, (3,3), padding=1)

        self.maxpooling = nn.MaxPool2d(kernel_size=(2,2), stride=2)

        self.fc = nn.Linear(1568, 10)

        self.activate = nn.ReLU()

        self.apply(weight_init)

    def forward(self, input):

        x = input

        x = self.activate(self.conv1(x))
        x = self.maxpooling(x)
        x = self.activate(self.conv2(x))
        x = self.maxpooling(x)
        x = self.activate(self.conv3(x))

        x = x.view(-1, 1568)

        x = self.fc(x)
```

```
114
115         return x
116
117 class ResidualBlock(nn.Module):
118     def __init__(self):
119         super(ResidualBlock, self).__init__()
120
121         # stride=1 is default
122         self.conv1 = nn.Conv2d(1, 8, (3,3), padding=1)
123         self.conv2 = nn.Conv2d(8, 16, (3,3), padding=1)
124         self.conv3 = nn.Conv2d(16, 32, (3,3), padding=1)
125
126         self.fc = nn.Linear(25088, 10)
127
128         self.activate = nn.ReLU()
129
130         self.apply(weight_init)
131
132         self.proj = nn.Conv2d(1, 32, kernel_size=1)
133
134     def forward(self, input):
135
136         x = input
137
138         # Residual connection
139         res = self.proj(x)
140
141         x = self.activate(self.conv1(x))
142         x = self.activate(self.conv2(x))
143         x = self.activate(self.conv3(x))
144
145         x = x + res
146
147         x = x.view(-1, 25088)
148
149         x = self.fc(x)
150
151         return x
152
153 class Regularization(nn.Module):
154     def __init__(self):
155         super(Regularization, self).__init__()
156
157         # stride=1 is default
158         self.conv1 = nn.Conv2d(1, 8, (3,3), padding=1)
159         #self.bn1 = nn.BatchNorm2d(8)                          # Batch normalization
160         self.conv2 = nn.Conv2d(8, 16, (3,3), padding=1)
161         #self.bn2 = nn.BatchNorm2d(16)                         # Batch normalization
162         self.conv3 = nn.Conv2d(16, 32, (3,3), padding=1)
163         #self.bn3 = nn.BatchNorm2d(32)                         # Batch normalization
164
165         self.dropout = nn.Dropout(p=0.5) # p=0.5 is default       # Dropout
166
167         self.fc = nn.Linear(25088, 10)
168
169         self.activate = nn.ReLU()
170
171         self.apply(weight_init)
172
```

```
173         self.proj = nn.Conv2d(1, 32, kernel_size=1)
174 
175     def forward(self, input):
176 
177         x = input
178 
179         # Residual connection
180         res = self.proj(x)
181 
182         #x = self.activate(self.bn1(self.conv1(x)))    # Batch normalization
183         #x = self.activate(self.bn2(self.conv2(x)))    # Batch normalization
184         #x = self.activate(self.bn3(self.conv3(x)))    # Batch normalization
185 
186 
187         x = self.activate(self.conv1(x))         # Dropout
188         x = self.activate(self.conv2(x))         # Dropout
189         x = self.activate(self.conv3(x))         # Dropout
190 
191         x = x + res
192 
193         x = x.view(-1, 25088)
194         x = self.dropout(x)                     # Dropout
195         x = self.fc(x)
196 
197         return x
198 
199 def cross_entropy(G, Y):
200     loss = F.cross_entropy(G, Y)
201     return loss
202 
203 def predict(net, X):
204     return net(X).argmax(dim=1)
205 
206 def accuracy(G, Y):
207     return (G.argmax(dim=1) == Y).float().mean()
208 
209 # initialize the test and training error statistics
210 test_accuracy = []
211 test_costs = []
212 test_iter = []
213 train_accuracy = []
214 train_costs = []
215 
216 # We also make these lists to store the results for every iteration
217 train_accuracy_batch = []
218 train_costs_batch = []
219 
220 # initialize the neural network and move it to the GPU if needed
221 # Here we can change between the different network implementations
222 
223 #net = NetFullyConnected()    # Exercise 1
224 #net = NetCNN()               # Exercise 2 and 3
225 #net = ResidualBlock()         # Exercise 4
226 net = Regularization()         # Exercise 5
227 net.to(device)
228 
229 # Code for calculating the number of parameters (exercise 2)
230 #with torch.no_grad():
231   #block = ["fc1", "fc2", "fc3"]
```

```
  #block = ["conv1", "conv2", "conv3", "fc"]
  #for mod in block:
    #module = getattr(net, mod)
    #parameters = [p.numel() for p in module.parameters()]
    #print(f"{mod}: {parameters}")

# SGD
#learning_rate = 0.01
#optimizer = optim.SGD(net.parameters(), lr=learning_rate, momentum=0.9)

# ADAM
learning_rate = 0.001
optimizer = optim.Adam(net.parameters(), lr=learning_rate)

# define the data loader for batches of the training data
batch_size = 30
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
↪  num_workers=2, shuffle=True)

num_epochs = 15

# move test data to the GPU if needed
X, Y = test_X.to(device), test_Y.to(device)

t = 0 # current iteration

patience = 0 # implemented for the stop criteria

start_time = time.time()
for epoch in range(num_epochs):
    net.train()
    for (batch_X, batch_Y) in trainloader:
        # move batch to the GPU if needed
        batch_X, batch_Y = batch_X.to(device), batch_Y.to(device)

        # zero the parameter gradients
        optimizer.zero_grad()

        batch_G = net.forward(batch_X)

        loss = cross_entropy(batch_G, batch_Y)

        loss.backward()

        # perform gradient descent step
        optimizer.step()

        # accumulate iterations
        t += 1

        # We put the results for each batch in the training cost/accuracy lists
        net.eval()
        with torch.no_grad():
            train_costs_batch.append(loss.item())
            train_accuracy_batch.append(accuracy(batch_G, batch_Y).item())

        net.train()

    net.eval()
```

```
    # We evaluate the performance on the test data after each epoch
    with torch.no_grad():
                G = net(X)
                test_costs.append(cross_entropy(G, Y).item())
                test_accuracy.append(accuracy(G, Y).item())
                test_iter.append(t)

                # the mean train accuracy/cost is added to the main lists and the batch lists are emptied
                train_accuracy.append(np.mean(train_accuracy_batch))
                train_costs.append(np.mean(train_costs_batch))
                train_accuracy_batch = []
                train_costs_batch = []

                print(f"Epoch {epoch+1:2d} - Step {t:5d}: train accuracy {100 * train_accuracy[-1]:6.2f}% " \
                      f"train costs {train_costs[-1]:5.2f}  " \
                      f"test accuracy {100 * test_accuracy[-1]:6.2f}% " \
                      f"test costs {test_costs[-1]:5.2f}")

    # Stop criteria
    if len(train_costs) > 2:
      if np.abs(train_costs[-2]-train_costs[-1])< 0.005:
        patience += 1
      if patience == 5:
        num_epochs = epoch + 1
        break

end_time = time.time()
elapsed = end_time - start_time

def training_curve_plot(title,
                        train_costs, test_costs, train_accuracy, test_accuracy,
                        batch_size, learning_rate, num_epochs, elapsed):

    lg=18
    md=13
    sm=9
    fig, axs = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle(title, y=1.15, fontsize=lg)
    elapsed_min, elapsed_sec = divmod(elapsed, 60)
    sub = f'|  Batch size:{batch_size}  |  Learning rate:{learning_rate} | Number of Epochs:{num_epochs} ' \
                + f'| Training time: {elapsed_min:.0f} min {elapsed_sec:.1f} sec |'
    fig.text(0.5, 0.99, sub, ha='center', fontsize=md)
    x = np.array(range(1, len(train_costs)+1))*num_epochs/len(train_costs)
    axs[0].plot(x, train_costs, label=f'Final train cost: {train_costs[-1]:.4f}')
    axs[0].plot(x, test_costs, label=f'Final test cost: {test_costs[-1]:.4f}')
    axs[0].set_title('Costs', fontsize=md)
    axs[0].set_xlabel('Epochs', fontsize=md)
    axs[0].set_ylabel('Cost', fontsize=md)
    axs[0].legend(fontsize=sm)
    axs[0].tick_params(axis='both', labelsize=sm)
    axs[1].plot(x, train_accuracy, label=f'Final train accuracy: {100*train_accuracy[-1]:.2f}%')
    axs[1].plot(x, test_accuracy, label=f'Final test accuracy: {100*test_accuracy[-1]:.2f}%')
    axs[1].set_title('Accuracy', fontsize=md)
    axs[1].set_xlabel('Epochs', fontsize=md)
```

```
    axs[1].set_ylabel('Accuracy (%)', fontsize=sm)
    axs[1].legend(fontsize=sm)
    axs[1].tick_params(axis='both', labelsize=sm)


# Below are different titles for the different learning curve plots

#title = "Training curve of the fully-connected network"
#title = "Training curve of the CNN"
#title = "Training curve of the CNN (ADAM)"
#title = "Training curve of the CNN with residual connection"
#title = "Training curve of the CNN with residual connection (no stop criteria)"
#title = "Training curve of the CNN with Batch normalization"
title = "Training curve of the CNN with Dropout"
training_curve_plot(title,
                    train_costs, test_costs, train_accuracy, test_accuracy,
                    batch_size, learning_rate, num_epochs, elapsed)

# Missclassifications
with torch.no_grad():
    # compute the predictions for the test set
    random_Yhat = predict(net, X)

    # sort the predictions with the incorrect ones first
    indices_incorrect_first = (random_Yhat == Y).float().argsort()

# plot the images
num_rows = 1
num_cols = 10
num_images = num_rows * num_cols
plt.figure(figsize=(num_cols, num_rows))

plotted_images = []

plot_position = 1

for index in indices_incorrect_first:

  if plot_position > num_images:
    break

  if Y[index].item() not in plotted_images:

    plt.subplot(num_rows, num_cols, plot_position)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(X[index, :, :].view(28, 28).cpu().numpy(), cmap=plt.cm.binary)

    # add the prediction as annotation (only incorrect ones, all in red)
    color = 'blue' if random_Yhat[index] == Y[index] else 'red'
    plt.text(0, 25, random_Yhat[index].item(), fontsize=25, color=color)

    plotted_images.append(Y[index].item())

    plot_position += 1

print(plotted_images)
plt.show()
```

```python

from sklearn.metrics import confusion_matrix
def plot_confusion_matrix(y_labels, y_preds):
    class_names = np.arange(10) # for MNIST labels
    cm = confusion_matrix(y_labels, y_preds).astype(np.float64)

    plt.figure(figsize=(6, 6))
    plt.imshow(cm, interpolation='nearest', cmap='coolwarm')
    plt.title("Confusion Matrix")
    plt.colorbar(shrink=0.75, fraction=0.05)

    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names)
    plt.yticks(tick_marks, class_names)

    plt.xlabel("Predicted label")
    plt.ylabel("True label")

    plt.tight_layout()
    plt.show()

with torch.no_grad():
    y_preds = predict(net, test_X.to(device)).cpu().numpy()
    y_labels = test_Y.to(device).cpu().numpy()

plot_confusion_matrix(y_labels, y_preds)
```